Histopathological Classification of Breast Cancer Images using Neural Networks

Deep Learning Final Project

by Group 6: Lourenço Passeiro (20221838), Miguel Marques (20221839), Peter Lekszycki (20221840), Tomás Gonçalves (20221894)

NOVA IMS, 15th December 2024

Abstract

In this project, we leverage deep learning techniques to assist in the diagnosis of breast cancer by classifying high-resolution histopathological images of breast tissue. By applying advanced pre-processing techniques and testing different types of models with different complexities, we achieved significant performance improvements. This report details our approach, experimental setup, and evaluation, highlighting the potential of AI to support medical diagnosis and decision-making.

1. Introduction

Breast cancer is one of the most common and fatal diseases affecting women around the world. Early and accurate diagnosis is crucial for effective treatment and improved patient outcomes. In this work, given the ethical challenge of defining the worst classification error (classifying a benign tumor as malignant, or the opposite), we did not privilege one over the other.

2. Methodology

The dataset is composed of three features: class, subclass, and magnification level, with the following characteristics:

- Around 70/30 class imbalance between malignant and benign samples;
- Very accentuated subclass imbalance, with Ductal Carcinoma possessing nearly half the samples, and the rest with proportions ranging from 5 to 12%;
- A very balanced distribution of magnification level.

2.1. Data Preprocessing

In this part, the aim was to make the dataset cleaner and more workable, while removing redundancy. The same procedures were applied in both projects' stages.

During importation, images were resized to 224x224 pixels, a size small enough to speed up computations but big enough to get satisfactory results, that was also used in the training of the pre-trained models utilized; the color space was modified from BGR to RGB; and the images normalized into [0,1]. Additionally, we removed duplicated images using the *cleanvision* library; imputed missing values, by simply looking at the images' path; and encoded the two target variables.

2.2. Image Transformations

This stage is crucial for emphasizing relevant features that can lead to improved image classification. The following types of transformations from *OpenCV* were applied and tested: color space changes (RGB, HSV, LAB), image denoising (Non-local means), gradient filters (Sobel, Laplacian), and edge detection (Canny Edge). [1-6]

2.3. Modelling

The same training, validation, and testing sets were used in the two stages, to ensure that both classes and subclasses were evenly represented and to avoid data leakage. However, depending on the specificities of each stage, modifications were done to each training set, such as data augmentation and undersampling. The hold-out method was applied, with a 70-15-15 split.

2.3.1 Stage 1: Binary Classification

As there was a big class imbalance for this problem, oversampling was applied in our training set, to have the same number of samples in both classes. For this, an ***ImageDataGenerator*** object from *Keras* was used with the following parameters: 30º rotation range, 20° width/height shift, zoom and brightness range, horizontal flips enabled, and filling missing pixels with the nearest existing ones. After data augmentation, the split became around 76% train, 12% val, and 12% test.

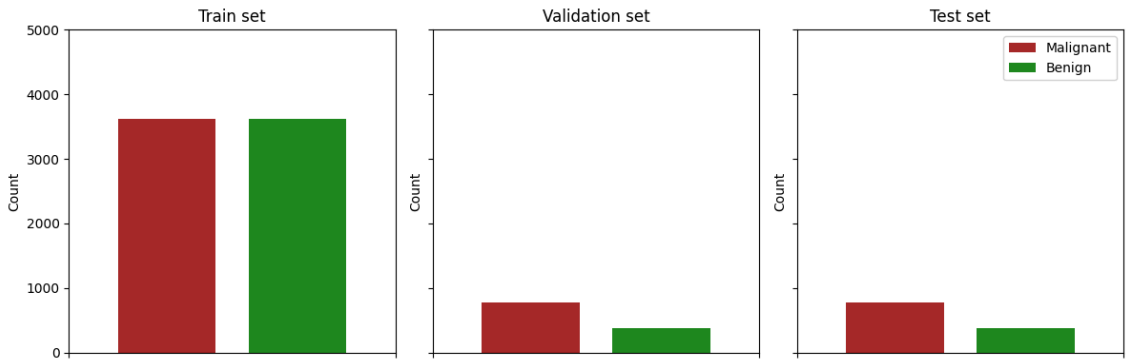

Fig. 1 - Class distribution of our training, validation, and testing set samples.

We chose to apply transfer learning, leveraging its predictive capabilities with smaller computational expenses compared to "hand-made" CNNs with similar results. *Keras* was the library used to build the model. From the varied models tested from *Keras Applications* [7], we chose the one that gave us the best results, while being very fast: ***VGG16***. This model has 5 convolutional blocks, and we opted to **fine-tune their last 2**. On top of that, we added **three densely-connected layers**, with **512**, **256**, and **128** nodes, all with ***reLU*** as the activation function, and a **dropout rate of 0.5** between them.

To train the model, we used three callbacks: ***EarlyStopping*** with a patience of 4; ***ReduceLROnPlateau*** with a patience of 3; and ***ModelCheckpoint***. ***Adam***, with a **learning rate decay schedule** applied, was the chosen optimizer, overperforming *RMSProp* in our tests. The model was trained during a maximum of **50 epochs**, with a **batch size of 64**.

The model achieved its best performance while training with **RGB images** without transformations for 35 epochs, with both training and validation accuracies being close to 90%. A **3% difference** between train and validation accuracy shows that the model is on the threshold of overfitting, on the good side.

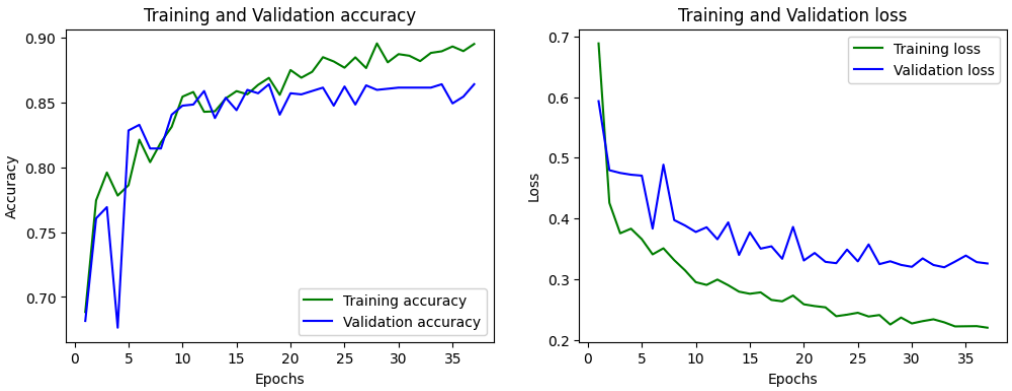

Fig. 2 - Training and validation loss and accuracy during model training

The model also had good results in the test set, with an **accuracy** and **weighted F1** of **85%**, and **AUC and AP values over 90%**. However, the **model is slightly biased towards the majority class**, with an accuracy on the minority class of only **70%**. The model also has a vague bias towards the '200X' factor and against '400X', but overall it generalizes well for every magnification level.

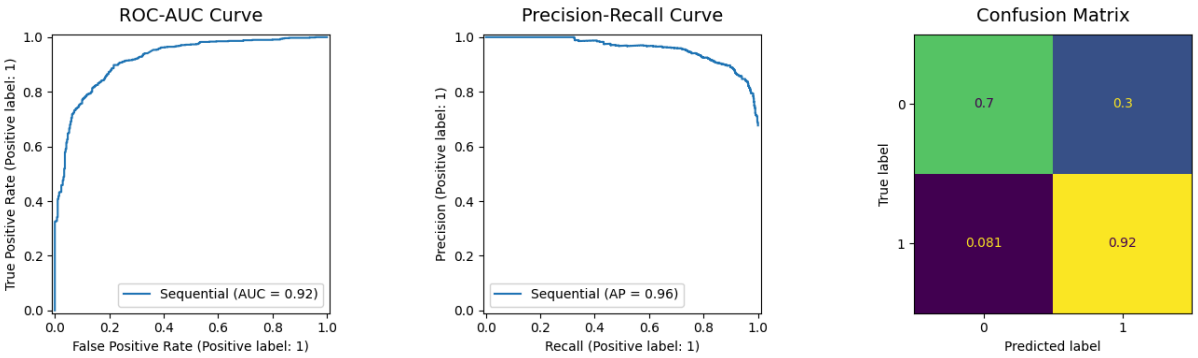

Fig. 3 - Model's performance on the test set, on ROC-AUC and precision-recall curves, and confusion matrix

2.3.2. Stage 2: Multiclass Classification

Similar to stage 1, there was a big class imbalance for this problem, with one of the eight classes having almost half of all samples. To combat this, the same oversampling procedure from Stage 1 was tested on minority classes. However, in this stage, undersampling was also tested, to avoid having to exponentially increase minority class sizes to match the number of the majority class, which would lead to the training of the model with a proportion of artificial images too high.

To achieve it, the following approach was applied [8]: embeddings were generated for each image; images were clustered based on their embeddings, and a redundancy score was computed by summing the cosine similarity of an image with all intra-cluster peers; finally, images were sorted based on their redundancy scores and the top *n* images (with highest scores) were marked for removal. By only performing comparisons intra-cluster, we are maintaining the diversity in the class, and removing the most redundant images.

Despite our best efforts to apply the sampling methods described above, the **best model performance was achieved without both oversampling and undersampling**, as applying them led to severe overfitting. Using class weights also led to the same problem.

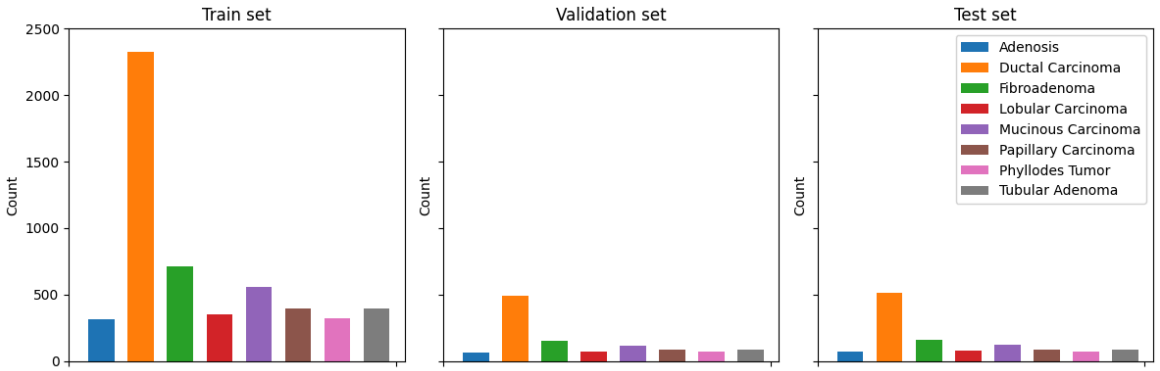

Fig. 4 - Subclass distribution of our training, validation and testing set samples.

Here, the ***VGG16*** model was once again applied, this time fine-tuning all the model's layers. This may seem counterproductive, as the main goal of transfer learning is taking advantage of pre-computed weights, but this was our way of personalizing the network to accept transformed images, different from the ones they were pre-trained with (RGB). Ideally, designing a CNN by hand would be the solution to this, but given the heavy computational limitations, it was not feasible. *Keras* was used to build the model.

On top of that, **3 densely-connected layers** were added, with **1024**, **512**, and **256** nodes respectively, all with ***reLU*** as the activation function, a ***Batch Normalization*** **layer** and **a dropout rate of 0.5** between them. The same callbacks and optimizer from the first stage were used. Models were trained for a maximum of 50 epochs, with **100 steps per epoch**, using **weighted f1-score** as the tracking metric, as opposed to the traditional accuracy, which was not the best option given the huge class imbalance that was not fixed with augmentation. All image transformations mentioned before were tested as input to the model, and the best performance came again using regular **RGB images**.

Despite our best efforts, trying multiple approaches and transformations, different from the final approach described above, **63%** was the best validation accuracy achieved, after **16 epochs**, with the model **underfitting**. Even though a 90%+ training accuracy was easily achievable, we tried very hard to suppress the initial huge difference between training and validation, rather than creating an obvious overfitting model (even though the model still **overfitted** at the end of training). The model got a **test accuracy** and **weighted F1** of **67%,** being very biased towards the majority class.

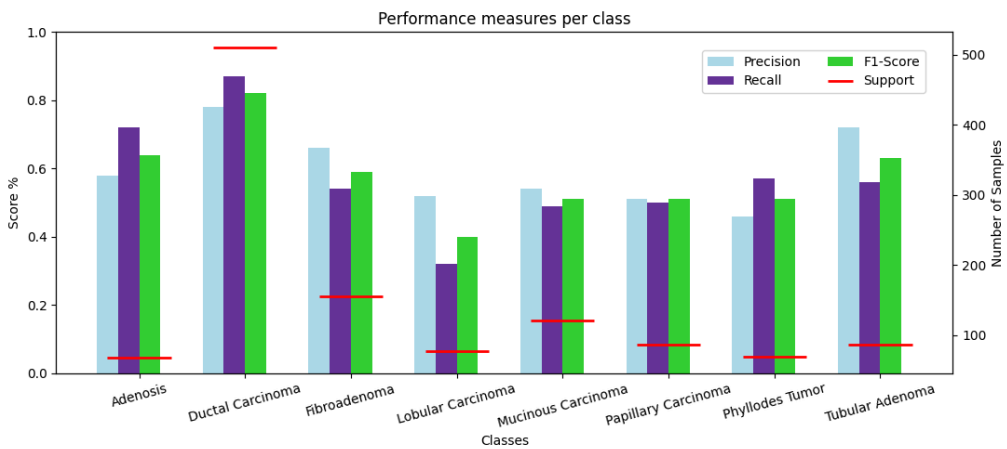

Fig. 5 - Model performance on the test set per class, on accuracy, precision, recall, and f1-score

The confusion matrix of the model shows us its clear bias toward the majority class, with an accuracy of **87%**, while the rest was around **50%**. As expected, the majority of prediction errors came when the model predicted the majority class, whereas the true class was another subclass from the same class (Malignant). This tells us that this model was also doing well at distinguishing between benign and malignant, and the issue clearly came when it came to subclasses (cancer types). This model also had worse results on the '400X' magnification level.

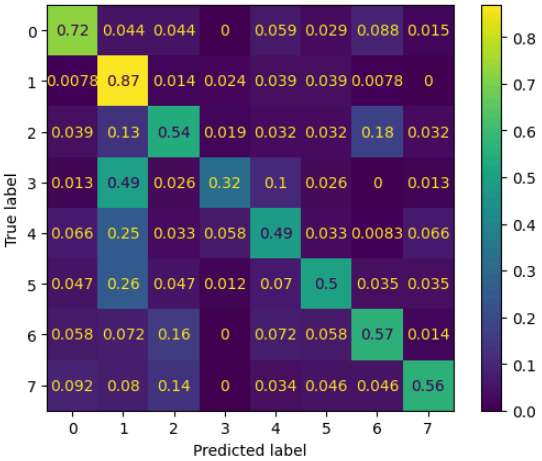

Fig. 6 - Confusion matrix of model's final predictions

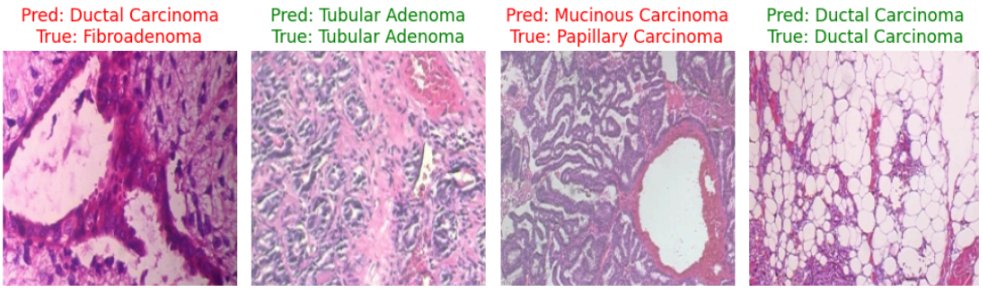

Fig. 7 - Examples of images predicted and their true labels

3. Conclusion

Although we ended this project with a bitter taste, as the final performance on the second stage was far from what we wanted, we feel that we have done everything in our power to achieve good results, testing the most varied approaches, while dealing with many computational issues. For instance, applying indispensable techniques like cross validation was unthinkable, given the already limited state of resources.

In a timeline with more time and computational resources, we would've liked to test applying feature extraction techniques to the images, in order to extract distinct meaningful attributes, which would later be combined into a single classifier, or an ensemble of models.

Nonetheless, during this project, we massively improved our programming and machine learning, by dealing with real-world images and their specificities.

4. Bibliography

Mauro Castelli, Yuriy Perezhohin: Deep Learning course materials

[1] https://docs.opencv.org/3.4/df/d9d/tutorial_py_colorspaces.html

[2] https://learnopencv.com/color-spaces-in-opencv-cpp-python/

[3] https://docs.opencv.org/3.4/d5/d69/tutorial_py_non_local_means.html

[4] https://docs.opencv.org/3.4/d5/d0f/tutorial_py_gradients.html

[5] https://docs.opencv.org/3.4/da/d22/tutorial_py_canny.html

[6]https://www.geeksforgeeks.org/feature-extraction-in-image-processing-techniques-and-applications/

[7] https://keras.io/api/applications/

[8] https://medium.com/@f.a.reid/image-similarity-using-feature-embeddings-357dc01514f8